

Projektaufgabe 3

Phase 1 – Das EDGE Modell (4 P)

Datenmanagement jenseits von Relationen

Gruppen Nummer (A17)

Acikyol Aleyna, 12032843

Özcan Ahmet, 12311287

Sadykova Nargiz, 12129337

June 9, 2026

Dieses Reporting Template dient der Vorbereitung der Abgabe von Phase 1.

EDGE Modell Import des Toy Beispiels (1 Punkte)

Beschreiben Sie kurz, welches Verfahren (inkl. Programmiersprache) Sie für den Import verwenden. Gab es Probleme beim Implementieren des Parsers? (0.5 P).

Für den XML-Import wurde Python mit der Standardbibliothek `xml.etree.ElementTree` verwendet. Das Dokument wird als String eingelesen, Sonderzeichen (Umlaute, `ß` etc.) werden vor dem Parsen per `str.replace()` normalisiert, da die DTD-Entities von `ElementTree` nicht automatisch aufgelöst werden. Anschließend wird der Baum in eine eigene `Node`-Klasse überführt und per Tiefensuche (*DFS*) in die Datenbank geschrieben.

Probleme gab es ausschließlich mit den HTML-Entities in den DBLP-Daten (z. B. `ü`), die `ElementTree` ohne DTD nicht kennt. Die Vorverarbeitung per `normalize_input()` löst dieses Problem zuverlässig.

Zeigen Sie (z.B. mittels Screenshot des Konsolenoutputs), dass Ihr Parser alle Elemente (und deren Inhalt) des Toy-Beispiels korrekt erkennt. (0.5 P).

Der folgende Ausschnitt zeigt den Konsolenoutput von `phase1_import.py` für das Toy-Beispiel (4 Publikationen):

```
=== EDGE-Modell Toy-Beispiel (Konsolenausgabe) ===
[bib] bib
  [venue] vldb
  [year] vldb_2023
  [article] SchmittKAMM23
    [author]: Daniel Ulrich Schmitt
    [author]: Daniel Kocher
    [author]: Nikolaus Augsten
    [author]: Willi Mann
    [author]: Alexander Miller
    [title]: A Two-Level Signature Scheme for Efficient Query Processing ...
    [pages]: 2686-2698
    [year]: 2023
```

```

        [volume]: 16
        [journal]: Proc. VLDB Endow.
        [number]: 11
        [ee]: https://doi.org/10.14778/3611479.3611480
        [url]: db/journals/pvldb/pvldb16.html#SchmittKAMM23
        [article] SchalerHS23
        [author]: ...
        ...
    [venue] sigmod
    [year] sigmod_2022
    [inproceedings] HutterAK0L22
    ...
    [inproceedings] ...
    ...

```

Alle Elemente und deren Inhalte werden korrekt erkannt. Die Attribute **mdate** und **orcid** werden wie gefordert ignoriert.

Geben Sie die Definition Ihrer Tabellen im Edge Modell an. Zeigen Sie (einen Ausschnitt) des Toy Beispiels nach dem Import als Edge Model in die DB (1 P).

Das EDGE-Modell besteht aus zwei Tabellen. Die **Node**-Tabelle speichert alle Knoten mit einem numerischen Primärschlüssel (**id**), einem symbolischen Bezeichner (**s_id**, z. B. Publikationsschlüssel oder Venue-Name), dem Knotentyp (**type**) sowie dem Textinhalt (**content**, nur für Blattknoten). Die **Edge**-Tabelle hält alle gerichteten Eltern-Kind-Kanten als Paare (**from**, **to**).

```

CREATE TABLE node (
    id          SERIAL PRIMARY KEY,
    s_id        VARCHAR,
    type        VARCHAR NOT NULL,
    content     TEXT
);

CREATE TABLE edge (
    "from"      INTEGER NOT NULL REFERENCES node(id),
    "to"        INTEGER NOT NULL REFERENCES node(id),
    PRIMARY KEY ("from", "to")
);

```

Ausschnitt des importierten Toy-Beispiels (erste Publikation, analog zu Tabelle 1 der Aufgabenstellung):

id	s_id	Node		Edge	
		type	content	from	to
0	bib	bib	null	0	1
1	vldb	venue	null	1	2
2	vldb_2023	year	null	2	3
3	SchmittKAMM23	article	null	3	4
4	null	author	Daniel Ulrich Schmitt	3	5
5	null	author	Daniel Kocher	3	6
6	null	author	Nikolaus Augsten	3	7
7	null	author	Willi Mann	3	8
8	null	author	Alexander Miller	3	9
9	null	title	A Two-Level Sig. [...]	3	10
...	...	...	...	...	...

Table 1: Ausschnitt des Toy-Beispiels nach Import ins EDGE-Modell

Berechnung der XPath Achsen im EDGE Modell (2 Punkte)

Die vier XPath-Achsen (`ancestor`, `descendant`, `following-sibling`, `preceding-sibling`) werden jeweils als rekursive SQL-Anfrage (WITH RECURSIVE) in der Datenbank berechnet. Python dient ausschließlich als Controller, der die Anfrage absetzt und das Ergebnis ausgibt.

Geben Sie für jede der geforderten Achsen, die Knotenmenge des Ergebnisses, sowie die Anzahl der Ergebnisknoten aus.

Achse	Ergebnisknoten IDs	Ergebnisgröße
ancestor (Daniel Ulrich Schmitt)	3, 2, 1, 0	4
descendants (vldb_2023, id=2)	3–16, 17–29	27
following SchmittKAMM23	17 (SchalerHS23)	1
preceeding SchmittKAMM23	(leer)	0
following SchalerHS23	(leer)	0
preceeding SchalerHS23	3 (SchmittKAMM23)	1

Table 2: Ergebnisse der XPath-Berechnung

Sofern Sie Annahmen zum Aufbau der Daten getroffen haben, geben Sie diese nachfolgend an:

- Die Dokumentreihenfolge wird durch die aufsteigenden `node.id`-Werte abgebildet, da alle Knoten in DFS-Reihenfolge (Vorbestellung) in die Datenbank eingefügt werden. `following-sibling` entspricht damit Geschwisterknoten mit `id > v_id`, `preceding-sibling` entsprechend `id < v_id`.
- Jeder Knoten hat höchstens einen Elternknoten (Baumstruktur), sodass die Elternsuche in der `edge`-Tabelle eindeutig ist.
- Die Attribute `mdate` und `orcid` werden beim Import ignoriert.

Zeitmanagement

Benötigte Zeit pro Person (nur Phase 1): **ca. 3 Stunden**